Bachelor Thesis

A Practical Evaluation of Oversmoothing and Oversquashing in Graph Neural Networks

Malaz Natouf

Learning on Graphs
RWTH Aachen University
Aachen, Germany
malaz.natouf@rwth-aachen.de

Professors

Prof. Dr. Christopher Morris
Prof. Dr. Michael Schaub

Supervisor

Timo Stoll

Registration Date

28 August 2026

Submission Date

Abstract:

Contents

1 Introduction **3**

1.1 Motivation 3

1.2 Research Questions 3

2 Background **3**

2.1 Message Passing and Receptive Fields 3

2.2 Depth versus Hidden Dimension 4

2.3 Homophily and Heterophily 6

2.4 Oversmoothing 8

2.5 PairNorm 10

3 Experimental Methodology **12**

3.1 Real-World Datasets 12

3.2 Evaluation Protocol 12

3.3 Oversmoothing Diagnostics 12

1 Introduction

1.1 Motivation

1.2 Research Questions

2 Background

2.1 Message Passing and Receptive Fields

A graph can be represented as $G = (V, E)$, where V is the set of nodes and E is the set of edges connecting pairs of nodes. In the graph-learning setting considered in this thesis, each node $v \in V$ also has an input feature vector $\mathbf{x}_v$. A graph neural network (GNN) uses these node features and the graph structure to create learned representations of the nodes. A widely used class of GNNs follows the *message-passing* principle, in which information is exchanged between adjacent nodes over a sequence of neural-network layers [GSR+17].

Let $\mathbf{h}_v^{(l)}$ denote the representation of node v after layer l, where $\mathbf{h}_v^{(0)} = \mathbf{x}_v$. In a message-passing layer, node v first receives information from the neighboring nodes in $\mathcal{N}(v)$. Messages generated from neighboring nodes are then combined using an aggregation operation. The result is then used to update the representation of node v. Following the message-passing framework of Gilmer et al. [GSR+17], this process can be written in the form

$$\mathbf{m}_v^{(l+1)} = \mathrm{AGG}^{(l)} \left(\left\{ M^{(l)} \left(\mathbf{h}_v^{(l)}, \mathbf{h}_u^{(l)}, \mathbf{e}_{vu} \right) \,\middle|\, u \in \mathcal{N}(v) \right\} \right), \tag{1}$$

followed by

$$\mathbf{h}_v^{(l+1)} = U^{(l)} \left(\mathbf{h}_v^{(l)}, \mathbf{m}_v^{(l+1)} \right). \tag{2}$$

Here, $M^{(l)}$ is the message function of layer l, $\mathrm{AGG}^{(l)}$ combines the messages arriving from the neighbors of v, and $U^{(l)}$ updates the representation of the target node. The optional vector $\mathbf{e}_{vu}$ represents features associated with the edge between nodes v and u. Since the neighbors of a node do not have a fixed order, the aggregation should not depend on the order in which these neighbors are presented. Common examples are sum, mean or maximum aggregation. The way these functions are used may differ depending on the GNN architecture [GSR+17, HYL17].

One such GNN model is GraphSAGE, which is extensively used in the experiments of this thesis and provides a concrete example of this procedure. At each layer, GraphSAGE first aggregates information from the immediate neighborhood of a node. The aggregated information is then combined with the node's own representation to obtain an updated node representation [HYL17]. Since the neighborhood representations used at one layer were themselves constructed in the preceding layer, information can propagate recursively through the graph. Thus, the first message-passing layer can incorporate information from direct neighbors. The second layer can additionally incorporate information from two edges away, and this process continues with increasing depth.

This recursive expansion motivates the notion of a *receptive field*. For a target node, the theoretical receptive field includes all nodes whose input information may influence its representation after a given number of message-passing layers. An L-layer network in which messages are passed between directly

connected nodes can in principle incorporate information from nodes that are at most L edges away from the target node. Therefore, adding more message-passing layers allows information to travel over a larger graph distance. Xu et al. describe the same principle through neighborhood aggregation. After k iterations, the representation can use structural information rooted at the target node up to depth k [XLT+18]. Figure 1 illustrates this expansion for successive message-passing layers.

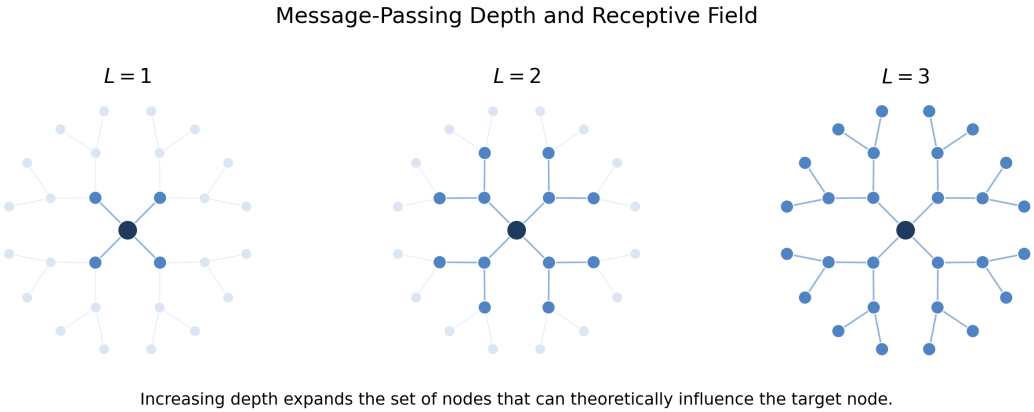


Figure 1: Receptive-field growth under repeated message passing. Each additional message-passing layer increases the maximum graph distance from which information can potentially reach the target node.

The theoretical receptive field should, however, be distinguished from the amount of information that actually influences a learned representation. For example, a node can be inside the L-hop neighborhood because a path to the target exists within L message-passing steps. However, this does not mean that all nodes in this neighborhood have the same influence. The learned parameters, aggregation operation, nonlinearities, graph structure and possible neighborhood sampling determine the strength of the influence from different nodes on the final representation. Xu et al. formalize this distinction by studying the sensitivity of a target representation to features at other nodes and show that the effective range of influence depends strongly on the local graph structure [XLT+18]. Therefore, two target nodes processed by networks of the same depth can have the same maximum hop range while exhibiting substantially different effective information propagation.

This distinction is central to the remainder of this thesis. Increasing depth provides a larger theoretical receptive field and therefore creates the possibility of incorporating information from more distant parts of a graph. It does not, by itself, guarantee that this information is preserved or used effectively. The following sections therefore separate the number of propagation steps from the capacity of the intermediate representations and subsequently examine how repeated message passing can affect the geometry and usefulness of node representations.

2.2 Depth versus Hidden Dimension

The previous section established that stacking message-passing layers increases the maximum graph distance over which information can potentially reach a target node. Network depth therefore determines how many propagation steps are possible. Nevertheless, the information that a graph neural network can represent is not determined by its depth alone. Another key architectural factor is the dimensionality of the intermediate node representations, often referred to as the *hidden dimension* or network width.

Using the notation from Section 2.1, let the representation of node v at layer l be $\mathbf{h}_v^{(l)}$. If the hidden representation contains p components, then

$$\mathbf{h}_v^{(l)} \in \mathbb{R}^p. \tag{3}$$

Network depth L and hidden dimension p describe two different aspects of the network. Depth specifies how many successive message-passing steps are applied. In architectures based on local one-hop message passing, greater depth means information can propagate over larger distances, thereby expanding the theoretical receptive field. Hidden dimension, on the other hand, specifies the size of the vector used to store and transform a node's intermediate representation. Therefore, increasing p does not add another propagation step, which means it does not extend the receptive field [Lou20].

This difference is illustrated in Figure 2. Increasing depth adds further message-passing layers, resulting in information passing through additional edges before the final representation is obtained. By contrast, increasing the hidden dimension enlarges the intermediate representation available at each layer. The two changes can therefore influence the model in different ways. Throughout this thesis, the term *hidden dimension* refers to the common dimensionality assigned to the intermediate hidden representations.

The distinction between these quantities is also relevant from an expressive-power perspective. Loukas studies message-passing graph neural networks under explicit restrictions on depth and width and shows that, for several graph problems in the considered theoretical setting, restricting these parameters limits the computational power of the network [Lou20]. The resulting bounds reveal a trade-off between depth and width for these problems, but they do not imply that the two architectural parameters are generally interchangeable. The theoretical analysis derives worst-case lower bounds under a defined computational model. Despite the identified trade-off, depth and width remain distinct architectural properties.

This difference is especially relevant when information from distant nodes must be preserved across several message-passing steps. Increasing the hidden dimension provides more space for intermediate representations but does not shorten the paths along which information travels. Conversely, increasing depth enables the network to gather information from nodes that are further away without changing the size of the hidden representations. Di Giovanni et al. analyze these effects separately in the context of long-range information propagation and show that increasing width and increasing depth can have fundamentally different effects on the sensitivity of message-passing neural networks to distant node information [DGGB$^+$23]. Their findings also highlight that increasing network depth cannot generally compensate for a limited representation width. Therefore, when comparing shallow and deep graph neural networks, it is important to distinguish between the impact of additional message-passing steps and changes in the dimensionality of node representations.

This thesis follows that distinction throughout the empirical analysis. Network depth is varied to investigate the effects of successive message-passing steps, whereas the hidden dimension is recorded or controlled independently. Separating these two architectural factors is necessary for understanding whether performance differences in deeper networks can be attributed to graph structure, representation geometry, or other characteristics of the learning task.

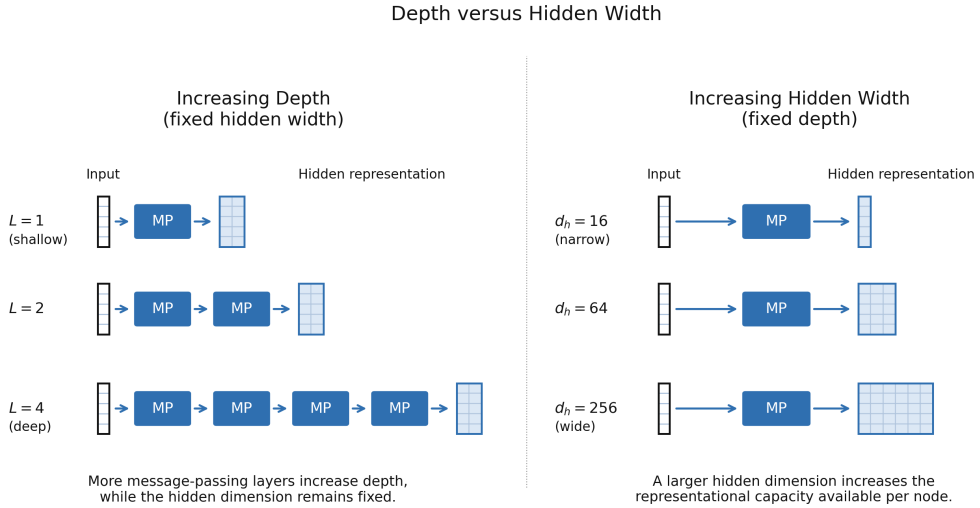


Figure 2: Conceptual distinction between depth and hidden dimension in a message-passing graph neural network. Increasing depth adds successive propagation steps and expands the theoretical receptive field, whereas increasing the hidden dimension enlarges the intermediate node representation without adding another message-passing step.

2.3 Homophily and Heterophily

The previous sections provided an overview of how message passing enables information to propagate between nodes in a graph and how network depth and hidden dimension influence different aspects of the GNN architecture. However, the usefulness of information aggregated from neighboring nodes also depends on the relation between the graph topology and the learning task. In node classification, this relation is often described in terms of *homophily* and *heterophily*.

In general, homophily describes a tendency of connected nodes to be similar, whereas heterophily describes a tendency of connected nodes to be dissimilar. The notion of similarity must be specified explicitly, since it may refer to class labels, node features, or other node attributes. In the graph-learning literature and in the experiments of this thesis, the term is primarily used with respect to class labels [ZYZ+20, ARE26]. Accordingly, a graph is more class-homophilous if connections frequently occur between nodes of the same class. In contrast, it is more class-heterophilous when edges frequently connect nodes belonging to different classes.

A simple measure of this property is the *edge homophily ratio*. For a graph $G = (V, E)$ with class label y_v assigned to each node v, it is defined as

$$h_{\text{edge}} = \frac{|\{(u, v) \in E \mid y_u = y_v\}|}{|E|}. \tag{4}$$

The value h_{edge} therefore represents the fraction of edges whose endpoints have the same class label [ZYZ+20, PKD+23]. Values close to one indicate that most edges are within classes, while values close to zero indicate that most edges connect nodes from different classes. Unless stated otherwise, the symbol h in this thesis refers to this raw class-label edge homophily measure.

The relation to message passing becomes intuitive when considering the aggregation mechanism introduced in Section 2.1. In a strongly homophilous graph, nodes typically aggregate information from neighbors that share the same class label. If their features also contain class-relevant information, the aggregation process can strengthen the signals relevant to the classification task. In a heterophilous graph, nodes often have neighbors belonging to different classes. Depending on the node features and task conditions, directly mixing these representations can make class-relevant information harder to separate. This observation helps explain why many graph neural network architectures and benchmarks have historically been associated with a homophily assumption and why specialized methods have been developed to address heterophilous graphs [ZYZ+20].

Figure 3 illustrates this distinction at the level of class labels. In the homophilous example, most edges connect nodes with the same class, whereas the heterophilous example contains a larger proportion of cross-class edges. The figure should be interpreted as an illustration of label–topology alignment rather than as a statement about task difficulty. In particular, the fact that neighboring nodes often belong to different classes does not imply that their relation is uninformative.

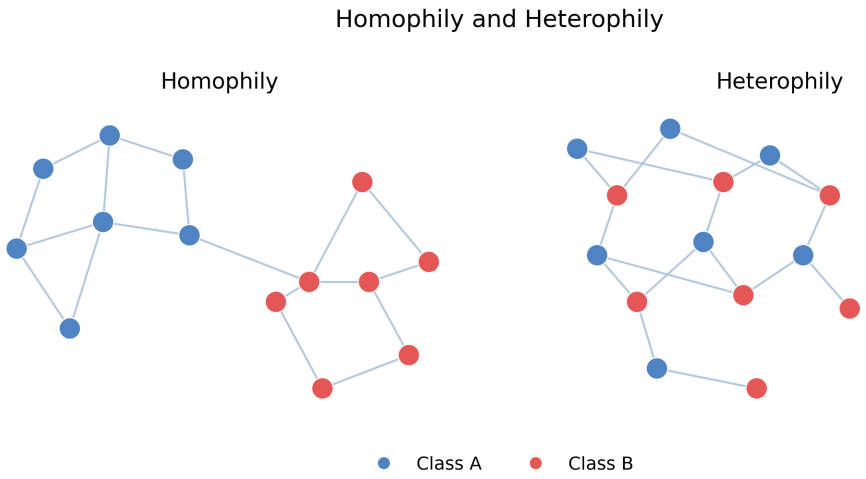


Figure 3: Schematic comparison of class-homophilous and class-heterophilous graph structure. In the homophilous example, edges predominantly connect nodes of the same class, whereas the heterophilous example contains a larger proportion of edges between different classes. The schematic illustrates label–topology alignment and does not imply that one of the two settings is inherently easier to learn.

This distinction is important because homophily describes the relationship between graph structure and the learning task rather than directly measuring how well a graph neural network will perform. Empirical studies have shown that standard GNNs can achieve strong predictive performance on some heterophilous graphs, while high homophily alone does not guarantee that a particular node-classification task is easy [PKD+23, ARE26]. The usefulness of neighborhood aggregation depends on additional factors, including the node features, the relationship between features and labels, the specific connectivity pattern, the learning objective, and the architecture used to process the graph.

A further limitation is that raw edge homophily should not be treated as an absolute measure that is directly comparable across arbitrary datasets. In particular, the fraction of same-class edges is affected by the number and relative sizes of the classes. Platonov et al. therefore distinguish raw edge homophily from adjusted measures that account for the amount of same-class connectivity expected from the class and degree distribution [PKD+23]. This thesis nevertheless uses raw edge homophily when a direct and

interpretable fraction of same-label edges is required, especially in the controlled experiments where target values of h are imposed explicitly. Adjusted homophily is used as a supporting diagnostic where appropriate rather than as a replacement for the controlled variable.

In this thesis, homophily and heterophily are therefore used to describe how labels align with graph structure, not as a sufficient explanation for model behavior. Arnaiz-Rodriguez and Errica similarly caution against the universal claim that homophily is inherently beneficial and heterophily inherently detrimental, emphasizing instead the interaction between the task, features, labels, and topology [ARE26]. This distinction is particularly relevant for the experiments presented later in this thesis, where homophily is varied under controlled conditions to examine whether differences in label–topology alignment are associated with changes in depth-related performance degradation and learned node representations.

2.4 Oversmoothing

Repeated message passing does not only enlarge the receptive field of a node. It also repeatedly mixes node representations with information from their neighborhoods. As discussed in the previous section, this mixing can be useful when neighboring nodes contain task-relevant information. However, repeated aggregation can cause node representations to become gradually similar across multiple layers. This phenomenon is commonly referred to as *oversmoothing* [LHW18, RBM23].

An early explanation of this effect was given by Li et al. [LHW18] for graph convolutional networks. They show that the propagation operation of a graph convolutional network can be understood as a form of Laplacian smoothing. Under this interpretation, each propagation operation combines a node's own representation with information from its neighbors. A moderate amount of such smoothing can be beneficial, as it encourages related nodes to develop more similar representations. On the other hand, excessive smoothing can eventually remove distinctions that are necessary for separating nodes, making it difficult to distinguish between them. Oversmoothing therefore refers to excessive representation smoothing rather than to neighborhood aggregation itself.

Let

$$H^{(l)} = \begin{bmatrix} (\mathbf{h}_1^{(l)})^\top \\ \vdots \\ (\mathbf{h}_{|V|}^{(l)})^\top \end{bmatrix}$$

denote the matrix containing all node representations at layer l. One way to measure how much the representations of neighboring nodes differ is through the graph Dirichlet energy. In a simplified unnormalized form, it can be written as

$$E\Big(H^{(l)}\Big) = \frac{1}{2} \sum_{(u,v)\in E} \left\| \mathbf{h}_u^{(l)} - \mathbf{h}_v^{(l)} \right\|_2^2. \tag{5}$$

Higher values indicate greater differences between the representations of neighboring nodes, whereas a value approaching zero indicates that representations connected by graph edges are becoming increasingly similar. Therefore, measures based on Dirichlet energy are widely used as diagnostics of oversmoothing [RBM23]. The exact normalized Dirichlet energy used in the experiments of this thesis is defined in a later section, since normalization is important when representation magnitudes differ across models or training conditions.

Several definitions and metrics for oversmoothing have been proposed in the literature. Rusch et al. [RBM23] provide a unified formulation in which oversmoothing is characterized by the layer-wise ex-

ponential convergence of a suitable node-similarity measure toward its collapsed value. This definition emphasizes that oversmoothing is not merely the observation that two neighboring representations happen to become slightly more similar. Rather, it describes a systematic loss of representation diversity as network depth increases. Other analyses characterize the limiting representations differently depending on the propagation operator. For example, Oono and Suzuki demonstrate that, under specific conditions on the graph spectrum and weight matrices, the representations in deep graph convolutional networks converge exponentially toward a restricted subspace [OS20]. In this subspace, the remaining information can be insufficient to differentiate between nodes that belong to the same connected component and have the same degree.

While these asymptotic results help explain why oversmoothing can occur, they do not suggest that every deep graph neural network will experience oversmoothing in the same way. The behavior depends on the propagation operator, graph topology, parameterization, normalization, and training dynamics. Recent work has further demonstrated that different oversmoothing diagnostics can lead to different conclusions for the same representations. In particular, Arnaiz-Rodriguez and Errica demonstrate that the raw Dirichlet energy can vary significantly due to simple rescaling, even when the separability of node embeddings remains largely unchanged [ARE26]. This motivates the use of multiple, scale-aware representation diagnostics rather than relying on a single metric.

Figure 4 illustrates the qualitative idea considered in this thesis. Node representations that are initially separated can move closer together as neighborhood information is repeatedly aggregated. The figure is meant to provide a simplified illustration of how node representations can become less diverse. It does not suggest that this process occurs in the same way for every graph, network architecture, or training setup.

A further distinction is essential for the empirical analysis in this thesis. Oversmoothing describes a change in the geometry or information content of node representations, but oversmoothing itself is not equivalent to a decrease in predictive accuracy. If all node representations become indistinguishable and lose their class-relevant information, a classifier can no longer distinguish between nodes that require different predictions. However, before complete representation collapse occurs, smoothing may reduce irrelevant differences between node representations while maintaining or even enhancing their class-related separability. Conversely, predictive performance can decrease for reasons other than oversmoothing, including optimization difficulties, an uninformative receptive field, or a mismatch between the graph structure and the learning task [KDL26, ARE26].

This distinction has received increasing attention in recent work. Kormann et al. report that methods designed to mitigate oversmoothing can preserve representation diversity without consistently producing corresponding improvements in predictive performance [KDL26]. Arnaiz-Rodriguez and Errica similarly argue that performance degradation should not be attributed to oversmoothing by default and emphasize that task-relevant representation separability is more directly linked to predictive performance [ARE26]. For this reason, the experiments in this thesis evaluate predictive performance and representation diagnostics separately. Evidence of representation collapse is considered an indication of oversmoothing only when it is considered together with the corresponding model performance.

Since repeated message passing can reduce important differences between node representations, several methods aim to preserve or restore their geometric structure. One such method is PairNorm, which was proposed specifically as a normalization-based approach for counteracting oversmoothing and is introduced in the following section.

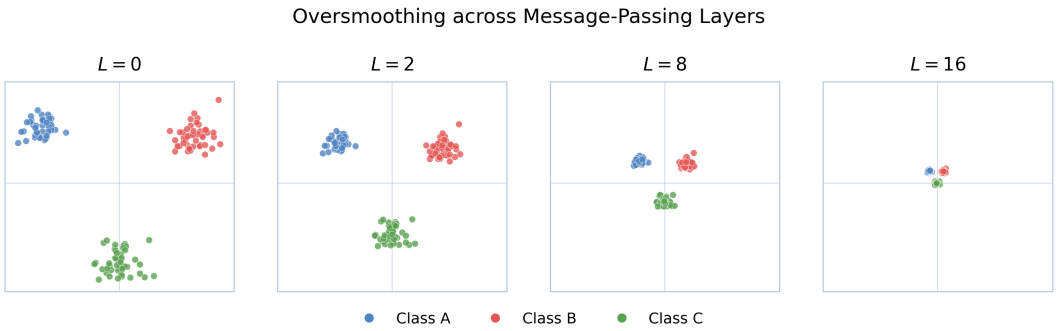


Figure 4: Conceptual illustration of oversmoothing under repeated message passing. Initially distinct node representations can become progressively more similar as neighborhood information is repeatedly aggregated. The schematic illustrates representation collapse and does not imply that all graph neural networks oversmooth at the same depth or rate.

2.5 PairNorm

The previous section described oversmoothing as a progressive loss of distinction between node representations under repeated message passing. This observation motivates normalization methods that do not prevent neighborhood aggregation itself, but constrain how strongly the geometry of the intermediate representations can contract. PairNorm is one such method. It was introduced by Zhao and Akoglu as a normalization layer specifically designed to counteract oversmoothing in graph neural networks [ZA20].

The main idea of PairNorm is to control the global spread of the node representations after a message-passing layer. Let $\widetilde{H} \in \mathbb{R}^{n \times p}$ denote the matrix of intermediate representations produced by such a layer, where n is the number of nodes and p the hidden dimension. Its i-th row, $\widetilde{\mathbf{h}}_i$, represents node i. Zhao and Akoglu motivate PairNorm through the observation that repeated graph convolution tends to decrease differences between node representations. They therefore propose constraining the total pairwise squared distance

$$\mathrm{TPSD}(H) = \sum_{i=1}^{n} \sum_{j=1}^{n} \|\mathbf{h}_i - \mathbf{h}_j\|_2^2 , \tag{6}$$

where TPSD denotes the total pairwise squared distance [ZA20]. Maintaining a non-vanishing global pairwise distance prevents all node representations from simultaneously collapsing toward the same point.

PairNorm implements this idea using two consecutive operations: *centering* and *scaling*. First, the mean representation across all nodes is removed. For each node i, the centered representation is

$$\widetilde{\mathbf{h}}_i^c = \widetilde{\mathbf{h}}_i - \frac{1}{n} \sum_{j=1}^{n} \widetilde{\mathbf{h}}_j . \tag{7}$$

This translation moves the mean of the node representations to the origin. Since the same vector is subtracted from every node, centering does not change the pairwise differences between node representations. Its purpose is instead to provide a common reference point from which the overall scale of the representation matrix can be controlled efficiently [ZA20].

The centered representations are then rescaled according to their average squared norm. The standard PairNorm transformation can be written as

$$\mathbf{h}_i^{\mathrm{PN}} = s \frac{\widetilde{\mathbf{h}}_i^c}{\sqrt{\frac{1}{n}\sum_{j=1}^{n} \left\| \widetilde{\mathbf{h}}_j^c \right\|_2^2}}, \tag{8}$$

where $s > 0$ is a scale hyperparameter [ZA20]. After this transformation, the representations remain centered and their total pairwise squared distance is fixed to a value determined by n and s. In particular, the original PairNorm derivation gives

$$\mathrm{TPSD}\left(H^{\mathrm{PN}}\right) = 2n^2 s^2. \tag{9}$$

PairNorm therefore does not prescribe which individual nodes should be close to or distant from one another. Instead, it constrains the global geometry of the complete set of node representations. Message passing can still make some representations more similar, but a simultaneous collapse of all node representations is prevented by maintaining their overall spread. This is important because PairNorm does not modify the graph topology, the receptive field, or the number of message-passing steps. It only acts on the intermediate representations produced by the network.

This geometric effect is illustrated in Figure 5. Repeated aggregation can initially move distinct node representations toward a more concentrated region of the embedding space. PairNorm first recenters these representations and after that it rescales their global spread. The figure should not be understood as suggesting that all distances between nodes are preserved or that their original positions are recovered. Rather, it visualizes the global constraint that prevents the complete representation set from contracting without bound.

The original formulation is computationally simple and introduces no additional learnable parameters. Zhao and Akoglu propose inserting the normalization between intermediate graph neural network layers, which allows PairNorm to be combined with different message-passing architectures [ZA20]. In this thesis, PairNorm is primarily combined with GraphSAGE so that the effect of repeated message passing can be compared with and without this normalization under otherwise matched architectural conditions. The exact model placement and experimental settings are specified later in the methodology.

However, it should be said that maintaining diversity among node representations does not necessarily lead to higher predictive accuracy. Zhao and Akoglu already observe that increased robustness against oversmoothing in deep graph neural networks does not necessarily improve absolute test performance when additional depth is not useful for the underlying task [ZA20]. Rusch et al. further demonstrate that a method can maintain a non-collapsed representation diagnostic, even as predictive performance declines with increasing network depth [RBM23]. PairNorm is therefore considered in this thesis as a method for modifying the geometry of node representations rather than a guaranteed solution to depth-related performance degradation.

This distinction also motivates a later component ablation. Since the standard PairNorm transformation consists of both centering and scaling, the experiments additionally evaluate a center-only and a scale-only operation to determine which component is associated with the observed changes in representation geometry and predictive performance. These two configurations are introduced specifically for the analysis in this thesis and are not presented as distinct methods in the original PairNorm work.

Finally, PairNorm is also reused later in the long-range information propagation experiments, where it continues to function as a non-structural normalization mechanism. Unlike graph rewiring, PairNorm does not introduce new edges or shorten graph distances. This makes it possible to examine whether

changes in long-range behavior necessarily require a structural modification of the graph, without re-defining PairNorm as an oversquashing-specific method.

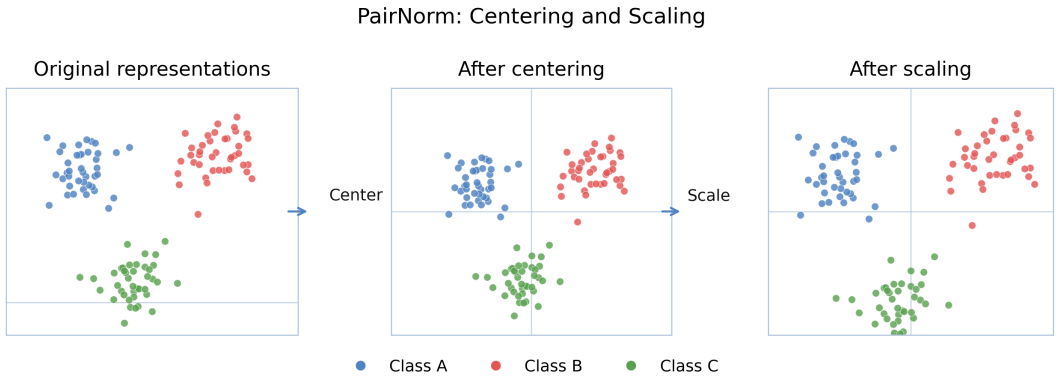


Figure 5: Conceptual effect of PairNorm on node-representation geometry. After message passing has reduced the global spread of the representations, PairNorm recenters them and rescales the representation matrix to a fixed global pairwise-distance scale. The operation constrains the overall geometry and does not preserve each individual pairwise distance.

3 Experimental Methodology

3.1 Real-World Datasets

3.2 Evaluation Protocol

3.3 Oversmoothing Diagnostics

References

[ARE26] Adrian Arnaiz-Rodriguez and Federico Errica. Oversmoothing, "Oversquashing", Heterophily, Long-Range, and More: Demystifying Common Beliefs in Graph Machine Learning. In *International Conference on Learning Representations*, 2026.

[DGGB+23] Francesco Di Giovanni, Lorenzo Giusti, Federico Barbero, Giulia Luise, Pietro Lio, and Michael M. Bronstein. On Over-Squashing in Message Passing Neural Networks: The Impact of Width, Depth, and Topology. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 7865–7885. PMLR, 2023.

[GSR+17] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural Message Passing for Quantum Chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272. PMLR, 2017.

[HYL17] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, volume 30, pages 1024–1034, 2017.

[KDL26] Niklas Kormann, Benjamin Doerr, and Johannes F. Lutzeyer. Position: Don't be Afraid of Over-Smoothing And Over-Squashing. *arXiv preprint arXiv:2601.07419*, 2026.

[LHW18] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper Insights Into Graph Convolutional Networks for Semi-Supervised Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 3538–3545, 2018.

[Lou20] Andreas Loukas. What Graph Neural Networks Cannot Learn: Depth vs Width. In *International Conference on Learning Representations*, 2020.

[OS20] Kenta Oono and Taiji Suzuki. Graph Neural Networks Exponentially Lose Expressive Power for Node Classification. In *International Conference on Learning Representations*, 2020.

[PKD+23] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. A Critical Look at the Evaluation of GNNs Under Heterophily: Are We Really Making Progress? In *International Conference on Learning Representations*, 2023.

[RBM23] T. Konstantin Rusch, Michael M. Bronstein, and Siddhartha Mishra. A Survey on Oversmoothing in Graph Neural Networks. Technical Report 2023-17, Seminar for Applied Mathematics, ETH Zurich, 2023.

[XLT+18] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. Representation Learning on Graphs with Jumping Knowledge Networks. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 5453–5462. PMLR, 2018.

[ZA20] Lingxiao Zhao and Leman Akoglu. PairNorm: Tackling Oversmoothing in GNNs. In *International Conference on Learning Representations*, 2020.

[ZYZ+20] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs. In *Advances in Neural Information Processing Systems*, volume 33, 2020.